DATABASE DESIGN DOCUMENTATION

Part 2: Logical Design

Case Study: Reliable Rentals

Prepared by:
Fabrisio Ponte Vela
Salome Collante Ramirez
Martineulr Marasigan

April 2026

1. Introduction

This document presents the logical data model for the Reliable Rentals vehicle rental management system, building directly upon the conceptual model developed in Part 1. The logical design phase translates conceptual entities and relationships into a formal relational schema, validates that schema through normalization to Third Normal Form (3NF), tests it against real-world user transactions, and establishes integrity constraints to ensure data accuracy and consistency.

The logical model serves as the blueprint for the physical database implementation. All steps are documented in the order required, with each step numbered for clarity. All assumptions are explicitly stated.

2. Case Study Overview

Reliable Rentals operates a fleet of vehicles rented to clients through multiple outlet locations. The system manages five core entities: Outlet, Vehicle, Client, Hire Agreement, and Staff. The logical design phase derives relations from these entities and their relationships, ensuring that the resulting relational schema is both normalized and transaction-ready.

Step 1: Derive Relations and Foreign Keys from the Conceptual Model

Relations are derived from the conceptual model by translating each entity into a table and determining where foreign keys (FKs) must be placed based on the multiplicity of each relationship. In a 1-to-many relationship, the FK is always placed on the 'many' side.

1.1 Relationship Analysis

Relationship	Multiplicity	FK Placement Decision
Outlet allocates Vehicle	1..1 : 1..*	outletNo placed as FK in Vehicle (many side)
Vehicle included in Hire Agreement	1..1 : 0..*	registrationNo placed as FK in Hire Agreement (many side)
Hire Agreement signed by Client	0..* : 1..1	clientNo placed as FK in Hire Agreement (many side)
Staff associated with Outlet	1..* : 1..1	outletNo placed as FK in Staff (many side)

1.2 Relational Schema

The relational schema is presented below. Primary keys are underlined and bold. Foreign keys are shown in italic red with the {FK} suffix. The schema reflects the FK placements derived from the relationship analysis above.

Outlet

Attribute	Key Type	Description
outletNo	Primary Key (PK)	Unique outlet identifier
address	—	Full postal address of the outlet
phone_number	Alternate Key (AK)	Contact phone number — must be unique
fax_number	Alternate Key (AK)	Fax number — must be unique

Vehicle

Attribute	Key Type	Description
registrationNo	Primary Key (PK)	Unique vehicle registration number
outletNo {FK}	Foreign Key → Outlet	Outlet the vehicle is currently allocated to
model	—	Vehicle model name
make	—	Vehicle manufacturer
engine_size	—	Engine displacement (decimal, > 0)
capacity	—	Passenger seating capacity (integer, >= 1)
current_mileage	—	Odometer reading (integer, >= 0)
daily_hire_rate	—	Cost per day to hire (decimal, > 0)

Client

Attribute	Key Type	Description
clientNo	Primary Key (PK)	Unique client identifier
fName	—	Client first name
lName	—	Client last name
home_address	—	Residential address (atomic field)
phone_number	—	Contact phone number
DOB	—	Date of birth (must be past date, client >= 18)
driving_license_number	Alternate Key (AK)	Driving license number — must be unique

Hire Agreement

Attribute	Key Type	Description
hireNo	Primary Key (PK)	Unique hire agreement identifier
clientNo {FK}	Foreign Key → Client	The client who signed this agreement
registrationNo {FK}	Foreign Key → Vehicle	The vehicle hired under this agreement
date_of_hire	—	Start date of the hire period
hire_termination_date	—	Expected end date of the hire period
mileage_before	—	Odometer reading at start of hire (>= 0)
mileage_after	— (NULLABLE)	Odometer reading at end of hire — nullable until vehicle is returned

Staff

Attribute	Key Type	Description
staffNo	Primary Key (PK)	Unique staff identifier
outletNo {FK}	Foreign Key → Outlet	Outlet this staff member is assigned to
fName	—	Staff first name
lName	—	Staff last name
home_address	—	Home address (atomic field)
phone_number	—	Home phone number
DOB	—	Date of birth
sex	—	Gender — restricted to 'M' or 'F'
company_start_date	—	Date joined the company (>= DOB + 16 years)
job_title	—	Role or position within the company
salary	—	Salary amount (decimal, > 0)

1.3 Compact Relational Schema Summary

Convention: Primary keys are underlined and bold. Foreign keys are shown in italics with {FK}. Alternate keys are noted separately.

```
Outlet(outletNo, address, phone_number, fax_number)
Vehicle(registrationNo, outletNo {FK}, model, make, engine_size,
capacity, current_mileage, daily_hire_rate)
Client(clientNo, fName, lName, home_address, phone_number, DOB,
driving_license_number)
HireAgreement(hireNo, clientNo {FK}, registrationNo {FK},
```

```
date_of_hire, hire_termination_date, mileage_before, mileage_after)
Staff(staffNo, outletNo {FK}, fName, lName, home_address,
phone_number, DOB, sex, company_start_date, job_title, salary)
```

Step 2: Validate the Logical Model Using Normalization to 3NF

Normalization is a systematic process of organizing a relational schema to reduce data redundancy and ensure data integrity. The model is validated progressively through First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

2.1 First Normal Form (1NF)

A relation is in 1NF if all attribute values are atomic (indivisible) and there are no repeating groups or multi-valued attributes.

Relation	1NF Assessment	Result	Notes
Outlet	All attributes (outletNo, address, phone_number, fax_number) are single-valued and atomic	**PASS**	
Vehicle	All attributes are single-valued and atomic. No repeating groups.	**PASS**	
Client	Name is decomposed into fName and lName per case study. Address treated as single atomic field per assumption.	**PASS**	Address assumption applied
Hire Agreement	All attributes atomic. Name decomposition and address atomicity follow same rules as Client.	**PASS**	
Staff	Name decomposed into fName and lName. All other attributes single-valued and atomic.	**PASS**	

Assumption: The case study does not decompose address into sub-fields (street, city, zip). Unlike name, which is explicitly split into first and last, address is treated as a single atomic string across all entities.

2.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key. Partial dependencies only occur when there is a composite primary key.

Relation	2NF Assessment	Result	Composite PK?

Outlet	Single-attribute PK (outletNo). No partial dependencies possible.	**PASS**	No
Vehicle	Single-attribute PK (registrationNo). All non-key attributes fully depend on it.	**PASS**	No
Client	Single-attribute PK (clientNo). No partial dependencies possible.	**PASS**	No
Hire Agreement	Single-attribute PK (hireNo). All attributes depend on the full PK.	**PASS**	No
Staff	Single-attribute PK (staffNo). No partial dependencies possible.	**PASS**	No

Since none of the five relations have a composite primary key, partial dependencies are structurally impossible. All relations pass 2NF without modification.

2.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key — that is, no non-key attribute depends on another non-key attribute.

Relation	3NF Assessment	Action	Result
Outlet	All non-key attributes (address, phone_number, fax_number) depend directly on outletNo. No transitive dependencies.	None	**PASS**
Vehicle	All attributes depend directly on registrationNo. No transitive dependencies.	None	**PASS**
Client	All attributes depend directly on clientNo. No transitive dependencies.	None	**PASS**
Staff	All attributes depend directly on staffNo. No transitive dependencies.	None	**PASS**
Hire Agreement	VIOLATIONS FOUND: hireNo → clientNo → client_name, client_address, client_phone hireNo → registrationNo → make, model These attributes are transitively dependent on hireNo through the FKs.	**Decompose**	**FIXED**

3NF Violation in Hire Agreement — Detailed Analysis

The original Hire Agreement relation contained redundant attributes that caused transitive dependencies:

- hireNo → clientNo → client_name, client_address, client_phone
  - These attributes describe the client, not the hire agreement itself. They are already stored in the Client relation.
- hireNo → registrationNo → make, model

- These attributes describe the vehicle, not the hire agreement. They are already stored in the Vehicle relation.

Resolution: The transitively dependent attributes are removed from Hire Agreement. They can always be retrieved through JOIN operations using the existing foreign keys (clientNo and registrationNo).

Before and After 3NF Decomposition

BEFORE (violates 3NF)	AFTER (compliant with 3NF)
HireAgreement(**hireNo,** *clientNo {FK},* *registrationNo {FK},* **client_name, ← VIOLATION** **client_address, ← VIOLATION** **client_phone, ← VIOLATION** **make, ← VIOLATION** **model, ← VIOLATION** date_of_hire, hire_termination_date, mileage_before, mileage_after)	**HireAgreement(** **hireNo,** *clientNo {FK},* *registrationNo {FK},* date_of_hire, hire_termination_date, mileage_before, mileage_after) *Removed attributes are retrieved via JOIN:* client_name, client_address, client_phone → JOIN Client make, model → JOIN Vehicle

2.4 Final 3NF Schema

After normalization, the final schema in 3NF is as follows:

Outlet(outletNo, address, phone_number, fax_number)

Vehicle(registrationNo, *outletNo {FK}*, model, make, engine_size, capacity, current_mileage, daily_hire_rate)

Client(clientNo, fName, lName, home_address, phone_number, DOB, driving_license_number)

HireAgreement(hireNo, *clientNo {FK}*, *registrationNo {FK}*, date_of_hire, hire_termination_date, mileage_before, mileage_after)

Staff(staffNo, *outletNo {FK}*, fName, lName, home_address, phone_number, DOB, sex, company_start_date, job_title, salary)

All five relations pass 3NF. The only modification was the removal of transitively dependent attributes from Hire Agreement.

Step 3: Validate the Logical Model Against 5 User Transactions

The following transactions validate that the normalized schema can support real-world queries. Each transaction states the query in plain English, identifies the tables involved, and explains the retrieval steps using entity relationships — without code.

Transaction 1: Most Recently Signed Hire Agreement	
Query	*What is the registration number, mileage before hire, and client full name for the most recently signed hire agreement?*
Tables Used	**Client, HireAgreement**
Steps	**1.** Identify the tables needed: HireAgreement (for hire details) and Client (for client name). **2.** Join HireAgreement to Client using the clientNo {FK} in HireAgreement, which references clientNo in Client. **3.** Select registrationNo and mileage_before from HireAgreement, and fName and lName from Client. **4.** Order the results by date_of_hire in descending order to place the most recent record first. **5.** Limit the result to 1 row to retrieve only the most recent hire agreement.

Transaction 2: Highest Daily Hire Rate Among Hired Vehicles	
Query	*What is the highest daily hire rate charged across all hire agreements?*
Tables Used	**Vehicle, HireAgreement**
Steps	**1.** Identify the tables needed: Vehicle (for daily_hire_rate) and HireAgreement (to restrict to hired vehicles only). **2.** Join HireAgreement to Vehicle using the registrationNo {FK} in HireAgreement, referencing registrationNo in Vehicle. This ensures only vehicles that have actually been hired are considered. **3.** Select daily_hire_rate from the Vehicle table for the matched records. **4.** Apply the MAX aggregate function to daily_hire_rate across all matched records to return the single highest value.

Transaction 3: High-Mileage Vehicles and Projected Rate Increase	
Query	*Which outlets have vehicles whose current mileage exceeds the average mileage of all vehicles hired in the last 2 months, and what would their updated daily hire rate be after a 5% increase?*
Tables Used	**Outlet, Vehicle, HireAgreement**
Steps	**1.** Identify the tables needed: Outlet (for outlet details), Vehicle (for mileage and rate), and HireAgreement (to filter recently hired vehicles). **2.** Join Vehicle to Outlet using outletNo {FK} in Vehicle, referencing outletNo in Outlet. **3.** Join HireAgreement to Vehicle using registrationNo {FK} in HireAgreement, to link hire records to vehicles.

	4. Filter HireAgreement records where date_of_hire falls within the last two months to isolate recently hired vehicles. **5.** Compute the average current_mileage of those filtered vehicles using the AVG aggregate function. **6.** Filter the Vehicle records for those whose current_mileage exceeds the computed average. **7.** For those vehicles, calculate the projected rate by multiplying daily_hire_rate by 1.05 and return it alongside outlet address and vehicle registration.

Transaction 4: Active Toyota Hire Agreements	
Query	*Toyota vehicles are being hired very often. Before importing more, how many clients currently have an active hire agreement for a Toyota vehicle, and what are the specific models involved?*
Tables Used	**Vehicle, Client, HireAgreement**
Steps	**1.** Identify the tables needed: Vehicle (for make and model), HireAgreement (for hire period and linking), and Client (for client identification). **2.** Join HireAgreement to Vehicle using registrationNo {FK} in HireAgreement, referencing registrationNo in Vehicle. **3.** Join HireAgreement to Client using clientNo {FK} in HireAgreement, referencing clientNo in Client. **4.** Filter Vehicle records where make = 'Toyota'. **5.** Filter HireAgreement records for active agreements: those where today's date is on or after date_of_hire AND on or before hire_termination_date. **6.** Count the distinct clientNo values from the filtered records to determine how many unique clients currently have an active Toyota hire. **7.** Select the distinct model values from the matched Vehicle records to identify which Toyota models are involved.

Transaction 5: Client Contact Lookup by Hire Number	
Query	*A client called but the call was dropped. They only provided the hire number AXNZN for a Mercedes Benz with the highest engine size. What are their contact details and address?*
Tables Used	**Client, HireAgreement, Vehicle**
Steps	**1.** Identify the tables needed: HireAgreement (to look up the hire record), Client (for contact details), and Vehicle (to verify the vehicle). **2.** Join HireAgreement to Client using clientNo {FK} in HireAgreement, referencing clientNo in Client. **3.** Join HireAgreement to Vehicle using registrationNo {FK} in HireAgreement, referencing registrationNo in Vehicle. **4.** Filter HireAgreement records for hireNo = 'AXNZN'. **5.** Using the clientNo {FK} matched in step 4, retrieve home_address and phone_number from the Client table for that specific client. **6.** The vehicle join confirms make = 'Mercedes Benz' and that the engine_size is the highest among all Mercedes Benz vehicles in the system.

Step 4: Define Integrity Constraints

Integrity constraints ensure that the data stored in the database is accurate, consistent, and meaningful. The following constraints are defined across six categories.

4.1 Primary Key Constraints

Each relation must have a primary key that is unique and never null. Primary keys uniquely identify every record in a relation.

Relation	Primary Key	Constraint
Outlet	**outletNo**	Must be unique and NOT NULL across all outlet records
Vehicle	**registrationNo**	Must be unique and NOT NULL — registration numbers are legally unique per vehicle
Client	**clientNo**	Must be unique and NOT NULL — system-assigned client identifier
Hire Agreement	**hireNo**	Must be unique and NOT NULL — each agreement has a distinct hire number
Staff	**staffNo**	Must be unique and NOT NULL — system-assigned staff identifier

4.2 Referential Integrity / Foreign Key Constraints

Foreign key constraints ensure that relationships between tables remain consistent. A foreign key value must match an existing primary key value in the referenced table, or be null where permitted.

Relation	Foreign Key	References	Constraint Description
Vehicle	outletNo {FK}	**Outlet(outletNo)**	A vehicle cannot be assigned to an outlet that does not exist in the Outlet table
HireAgreement	clientNo {FK}	**Client(clientNo)**	A hire agreement cannot reference a client that does not exist in the Client table
HireAgreement	registrationNo {FK}	**Vehicle(registrationNo)**	A hire agreement cannot reference a vehicle that does not exist in the Vehicle table
Staff	outletNo {FK}	**Outlet(outletNo)**	A staff member cannot be assigned to an outlet that does not exist in the Outlet table

4.3 Alternate Key Constraints

Alternate keys are candidate keys that were not chosen as the primary key. They must still be unique within their respective relations.

Relation	Alternate Key	Constraint
Client	driving_license_number	Must be UNIQUE — no two clients can share the same driving license number
Outlet	phone_number	Must be UNIQUE — no two outlets can share the same phone number
Outlet	fax_number	Must be UNIQUE — no two outlets can share the same fax number

4.4 Required Data Constraints

Required data constraints specify which attributes must always contain a value (NOT NULL) and which are permitted to be null under defined conditions.

Relation	Attribute	Required?	Justification
All Relations	All attributes (general)	**YES (NOT NULL)**	All attributes are required unless explicitly stated otherwise
HireAgreement	mileage_after	**NO (NULLABLE)**	Mileage after hire is only known when the vehicle is returned; unknown at agreement creation time

4.5 Attribute Domain Constraints

Domain constraints restrict the set of valid values that can be stored in each attribute.

Vehicle

Attribute	Domain Constraint
engine_size	Must be a positive decimal value greater than 0 (e.g., 1.6, 2.0)
capacity	Must be a positive integer greater than or equal to 1 (at least 1 seat)
current_mileage	Must be an integer greater than or equal to 0
daily_hire_rate	Must be a positive decimal value greater than 0

Client and Staff

Attribute	Applies To	Domain Constraint

DOB	Client, Staff	Must be a past date. For Clients: must be at least 18 years before the date_of_hire on any agreement they sign

Staff

Attribute	Domain Constraint
sex	Restricted to 'M' or 'F' — any other value is invalid
salary	Must be a positive decimal value greater than 0
company_start_date	Must be greater than or equal to DOB + 16 years — a staff member cannot have started before the minimum working age

Hire Agreement

Attribute	Domain Constraint
date_of_hire	Must be less than or equal to hire_termination_date
hire_termination_date	Must be greater than or equal to date_of_hire — the end date cannot precede the start
mileage_before	Must be an integer greater than or equal to 0
mileage_after	Must be greater than or equal to mileage_before — a vehicle cannot return with fewer miles than it left with

4.6 General Constraints

General constraints enforce business rules that span multiple attributes or relations and cannot be captured by simple domain or referential constraints alone.

#	Constraint	Description
1	**One active hire per vehicle**	A vehicle can be part of only one active hire agreement at any time. An agreement is active when today's date falls between date_of_hire and hire_termination_date (inclusive).
2	**Minimum staff per outlet**	Each outlet must have at least one staff member assigned to it at all times. An outlet without any staff violates the 1..* multiplicity of the Staff–Outlet relationship.
3	**Mileage consistency**	The mileage_before recorded on a new hire agreement must match the current_mileage in the Vehicle table at the time the agreement is created. After the hire concludes, current_mileage in Vehicle must be updated to reflect mileage_after.

Step 5: Entity-Relationship Diagram — Logical Level

The logical-level E-R diagram extends the conceptual model by including foreign keys as explicit attributes within each entity. This diagram reflects the final 3NF schema and shows how relations are physically linked through their foreign key references.

Key Differences from Conceptual E-R Diagram

Conceptual Level (Part 1)	Logical Level (Part 2)
No foreign keys as attributes	Foreign keys explicitly shown as attributes (e.g., outletNo {FK} in Vehicle)
Relationships shown as diamonds	Relationships shown as connecting lines with FK attributes
Entities only have own attributes	Entities include FK attributes linking to parent relations
Hire Agreement had redundant attributes	Hire Agreement is normalized (3NF): client/vehicle details removed

Logical E-R Diagram

The diagram below was generated in Lucidchart and shows all five entities with their full attribute lists including foreign keys. Primary keys are underlined, foreign keys are marked with {FK}.

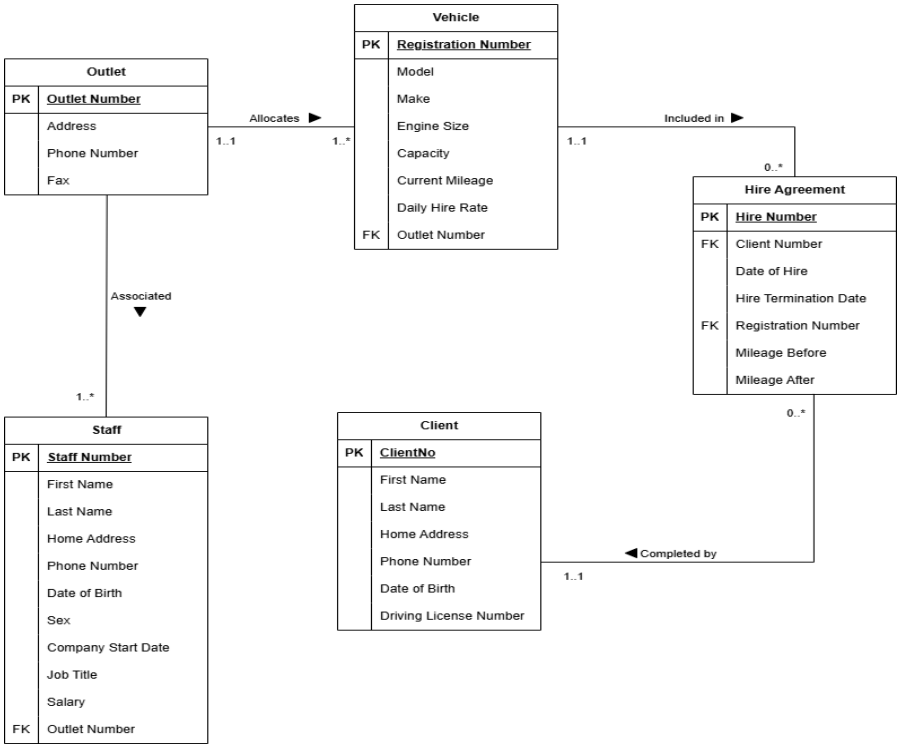

Figure 1: Logical E-R Diagram for Reliable Rentals Database (generated in Lucidchart). Replace this placeholder with the exported diagram image.

Entity Attribute Summary for Diagram Reference

Entity	Primary Key	All Attributes (incl. FKs)
Outlet	**outletNo**	outletNo, address, phone_number, fax_number
Vehicle	**registrationNo**	registrationNo, outletNo {FK}, model, make, engine_size, capacity, current_mileage, daily_hire_rate
Client	**clientNo**	clientNo, fName, lName, home_address, phone_number, DOB, driving_license_number
HireAgreement	**hireNo**	hireNo, clientNo {FK}, registrationNo {FK}, date_of_hire, hire_termination_date, mileage_before, mileage_after
Staff	**staffNo**	staffNo, outletNo {FK}, fName, lName, home_address, phone_number, DOB, sex, company_start_date, job_title, salary

Assumptions

The following assumptions were made during the logical design phase, in addition to those established in Part 1:

#	Assumption	Rationale
1	**One outlet per vehicle at a time**	Retained from Part 1. Enforced by outletNo {FK} in Vehicle.
2	**Client may exist without a hire agreement**	Retained from Part 1. Supported by 0..* participation of Client in HireAgreement.
3	**A vehicle may not always be hired**	Retained from Part 1. Supported by 0..* on the HireAgreement side.
4	**Each agreement is per one vehicle and one client**	Enforced by the single clientNo {FK} and registrationNo {FK} in HireAgreement.
5	**Address is a single atomic field**	The case study does not decompose address. Unlike name (which is split into fName/lName), address is stored as one atomic string field across Outlet, Client, Staff, and HireAgreement (pre-3NF).
6	**mileage_after is the only nullable field**	All other attributes are required at record creation. mileage_after cannot be known until the vehicle is returned.
7	**Redundant hire agreement attributes removed**	client_name, client_address, client_phone, make, and model were removed from HireAgreement during 3NF normalization. They are retrievable via JOINs to Client and Vehicle.

Summary

This document has presented the complete logical data model for the Reliable Rentals vehicle rental system, addressing all five required design steps:

- Step 1 — Relations and foreign keys were derived from the conceptual model. The relational schema was defined for all five entities with appropriate FK placement based on relationship multiplicity.
- Step 2 — The schema was validated through normalization to 3NF. All relations passed 1NF and 2NF without modification. The Hire Agreement relation was modified to remove transitively dependent attributes (client name/address/phone and vehicle make/model), resulting in a fully normalized 3NF schema.
- Step 3 — Five user transactions were defined in plain English and validated against the schema, demonstrating that all required queries can be answered using JOIN operations through the established foreign key relationships.
- Step 4 — Integrity constraints were defined across six categories: primary keys, referential integrity, alternate keys, required data, attribute domain constraints, and general business rules.
- Step 5 — The logical-level E-R diagram was generated in Lucidchart, showing all entities with their full attribute lists including explicit foreign key attributes, primary key underlines, and normalized structure.

All assumptions made during the design process are documented in the Assumptions section. This logical model is ready to serve as the direct specification for physical database implementation in Part 3.